Mathematical Foundations and Experimental Implementation of an Extended Kalman FIlter for Mobile Robot Localization

Diego Macías Blanco

June 2026

1 Abstract

Accurate robot localization is essential for autonomous mobile robots, as errors in wheel odometry can accumulate during motion and lead to deviations from the desired trajectory. This work presents the implementation and experimental evaluation of an Extended Kalman Filter (EKF) for planar pose estimation on a differential-drive mobile robot. The proposed approach combines wheel-encoder odometry and Inertial Measurement Unit (IMU) gyroscope measurements. Encoder-derived linear and angular velocities are used in the prediction stage, while the angular velocity measured by the gyroscope is integrated to obtain an orientation estimate used in the correction stage. The filter was evaluated using two reference paths, rectangular and zigzag trajectories, on a real robot. Experimental results were compared against encoder-only odometry through trajectory plots and Euclidean positional errors measured at five control points. The EKF achieved a reported reduction of 27.01% in the final positional error for the rectangular trajectory and 65.41% for the zigzag trajectory. These results demonstrate that incorporating gyroscope measurements into the estimation process can improve the robot's positional estimation, particularly during trajectories involving frequent changes in direction.

Keywords: Extended Kalman Filter, IMU, Odometry, Covariance, Prediction, Correction

2 Introduction

Accurate state estimation is a fundamental requirement for autonomous mobile robots, as uncertainties in wheel odometry and inertial measurements can lead to errors in the estimated robot pose. In this work, an Extended Kalman Filter (EKF) is implemented on a differential-drive mobile robot to estimate its planar position and orientation by combining wheel-encoder odometry with gyroscope measurements from an Inertial Measurement Unit (IMU). The encoder-derived linear and angular velocities are used in the prediction stage to propagate the robot's state, while the IMU's raw angular velocity measurement is integrated to obtain an independent orientation estimate, which is subsequently used in the correction stage. This approach aims to reduce orientation-estimation uncertainty while maintaining a continuous estimate of the robot's planar position.

Kalman filter assumes linear models as given in [1]. It is considered the Bayes Filter for the Gaussian linear case. It performs recursive state estimation including:

1. Prediction step: exploit the controls (Where are we?).
2. Correction step: exploit the observations.

$$Belief = Prediction + Correction$$

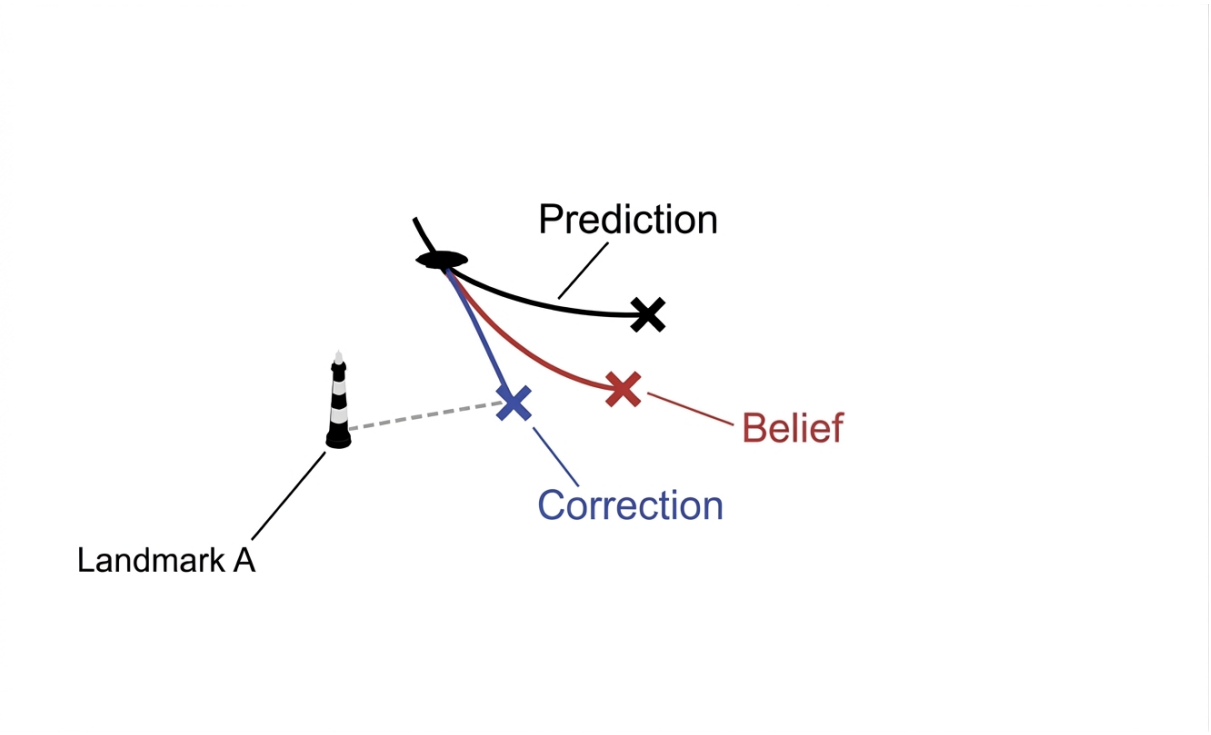


Figure 1: Kalman Filter Localization Model adapted from [2]

3 Theoretical Framework

3.1 Probability

The first step is to learn three basic rules of probability assuming they are all independent cases:

- Joint probability: $P(A, B) = P(A) \cdot P(B)$
  - Ex) The probability of getting a number 2 from one dice and number 4 from another dice: $P(2, 4) = P(2) \cdot P(4) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$
- Conditional probability: $P(A|B) = \frac{P(A,B)}{P(B)}$
  - Ex) The probability of getting a king card when having facecards. In this example, a king card will always be a facecard, so this events are not independent. As there are only 4 king cards in the maze, $P(A, B) = \frac{4}{52}$. On the other hand, the probability of getting a facecard (J, Q, K) is $\frac{12}{52}$. So, $\frac{P(A,B)}{P(B)} = \frac{\frac{4}{52}}{\frac{12}{52}} = \frac{4}{12} = \frac{1}{3}$
- Total probability: $P(A) = \sum_B P(A|B) \cdot P(B)$
  - Ex) The probability of getting late at school $P(A)$ taking into account several means of transportation:

    $P(A) = P(late) = P(late|car) \cdot P(car) + P(late|bus) \cdot P(bus) + P(late|walk) \cdot P(walk) = (0.1)(0.5) + (0.3)(0.3) + (0.05)(0.2) = 0.05 + 0.09 + 0.01 = 0.15$

Table 1: Marginal Probability B

Arriving by	$P(B)$
Car	0.5
Bus	0.3
walk	0.2

Table 2: Conditional Probability $A|B$

Arriving late because of traffic	$P(A\|B)$
P(late|car)	0.1
P(late|bus)	0.3
P(late|walk)	0.05

3.2 Bayes Rule

Bayes' rule provides a mathematical framework for updating the probability of a hypothesis when new evidence becomes available. It is expressed as

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

In the context of inference, this can be interpreted as

$$P(Cause|Effect) = \frac{P(Effect|Cause)}{P(Effect)}P(Cause)$$

For example, consider the following hypothesis and observation:

- **Cause**: You have the flu.
- **Effect**: You have a fever.

In this case, the fever is the observed evidence, and the objective is to infer the probability that the person has the flu.

The Four Probabilities

The four quantities involved in Bayes' rule can be interpreted as follows:

Term	**Meaning**	**Value**
$P(flu)$	Prior: probability you have the flu	0.01
$P(fever \mid flu)$	Likelihood: probability of fever given you have the flu	0.90
$P(fever)$	Evidence: probability of fever in the general population	0.05
$P(flu \mid fever)$	Posterior: probability you have the flu **given** you have a fever	Unknown

The posterior probability can then be calculated using Bayes' rule:

$$P(flu \mid fever) = \frac{P(fever \mid flu)\, P(flu)}{P(fever)}$$

Substituting the given values,

$$P(\text{flu} \mid \text{fever}) = \frac{0.90 \times 0.01}{0.05} = \frac{0.009}{0.05} = 0.18$$

Therefore, given the assumed probabilities, the posterior probability of having the flu given that a fever has been observed is 18%.

3.3 Bayes Filter

The Bayes filter extends Bayes' rule to sequential state estimation. It provides a recursive framework for estimating the state of a system from noisy control inputs and observations.

The Bayes Filter consists of two main steps:

1. **Prediction**: Imagine you give 1 step forward (control input). You think "I must be near the couch", but you are not sure because you were walking with your eyes closed.
2. **Correction**: You extended your arm and reached a wooden surface (observation). The correction step multiplies your prediction by the probability of touching wood = stand by side of the table. The belief becomes precise.

Lets begin with the prediction step. We want to predict the probability distribution of our next position (x_{t+1}) depending on an observation (z_t) and our control input (u_t):

$$P(x_{t+1}|z_t, u_{t+1})$$

We can use the Law of Total Probability for getting the probability of our next position (x_{t+1}), but instead of using B as the mean of transportation as on the example showed above, we use B as the robot's position depending on the time (x_t). Because the robot can be at any position (3.2 m, 3.21 m, 3.215 m), this variable is not discrete anymore and becomes continuous. We apply the Continuous Law of Total Probability being $A = x_{t+1}|z_t, u_{t+1}$ and $B = x_t$:

$$P(A) = \int P(A|B)P(B)\,dB$$

$$P(x_{t+1}|z_t, u_{t+1}) = \int P(x_{t+1}|x_t, z_t, u_{t+1})P(x_t)\,dx$$

Now, if we use $P(B) = P(x_t)$ we are not considering the observation and control inputs for that moment, which will only be the Prior state before moving or sensing. We add (z_t, u_t) to consider our actual Belief of where we are ($P(B) = P(x_t|z_t, u_t)$) and we get:

$$P(x_{t+1}|z_t, u_{t+1}) = \int P(x_{t+1}|x_t, z_t, u_{t+1})P(x_t|z_t, u_t)\,dx$$

The Bayes Filter incorporates the Markov Chain model that assumes that if we want to predict the future in the sequence, all that matters is the current state: $P(q_i = a|q_1...q_{i-1}) = P(q_i = a|q_{i-1})$

In our case, if we know our current position (x_t) and the next control input (u_{t+1}) that will lead to my next position (x_{t+1}), that future state has nothing to do with our current observation (z_t). Only the current state needs to know about our current observations as we did when calculating $P(B)$. That is why we can take out (z_t) from the equation:

$$P(x_{t+1}|z_t, u_{t+1}) = \int P(x_{t+1}|x_t, u_{t+1})P(x_t|z_t, u_t)\,dx$$

Now each prediction is called a belief: $\overline{bel}(x_{t+1}), \overline{bel}(x_t), \overline{bel}(x_{t-1})$. And because:

$$\overline{bel}(x_{t+1}) = \int P(x_{t+1} \mid x_t, u_{t+1})\, bel(x_t)\, dx$$

We can calculate the prediction of $\overline{bel}(x_t)$ as:

$$\boxed{\text{Prediction: } \overline{bel}(x_t) = \int P(x_t|x_{t-1}, u_t) bel(x_{t-1})\, dx}$$

And that is a recursive model!!!

That is just the belief for our prediction based on our current state and control inputs. For the correction, we need to include the probability of the observation at that future state:

$$\boxed{\text{Correction: } bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)}$$

Where n is just a normalization constant so that the sum of the probabilities is equal to 1. Therefore, the Bayes filter recursively alternates between prediction and correction. This recursive structure allows the robot to continuously estimate its state as new control inputs and sensor observations become available.

3.4 Gaussian Distribution

Because a random variable cannot be fully predicted, its behavior is characterized by a probability distribution. The Gaussian distribution is a mathematical function that describes the probability distribution of a random variable using two main parameters:

1. μ **(mean)**: the most probable value for a Gaussian distribution.
2. σ **(standard deviation)**: a measure of the dispersion of values around the mean.

The Gaussian probability density function is given by:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

4 Kalman Filter

4.1 Gaussian Assumptions

We assume that the state distribution follows a Gaussian distribution as given in [3]. Given a probability distribution of the state vector

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}^T,$$

we assume the following properties:

1. **Marginal distributions:**
$$p(x_a) \sim \mathcal{N}, \qquad p(x_b) \sim \mathcal{N}$$
2. **Conditional distributions:**
$$p(x_a \mid x_b) \sim \mathcal{N}, \qquad p(x_b \mid x_a) \sim \mathcal{N}$$

We also assume that the world is linear. Therefore, any transformation from one state to another can be described using a linear function:

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$$

A Gaussian distribution transformed through a linear function remains Gaussian

4.2 Linear System Models

The linear models used to describe a dynamic system are divided into two main components.

4.2.1 State Transition Model

The state transition model describes how the system evolves from one state to another:

$$\mathbf{x}_t = A_t\mathbf{x}_{t-1} + B_t\mathbf{u}_t + \boldsymbol{\omega}_t$$

Where:

- $\mathbf{x}_t$: Current state vector.
- $\mathbf{x}_{t-1}$: Previous state vector.
- A_t: Matrix describing the evolution of the system.
- $\mathbf{u}_t$: Control input.
- B_t: Matrix describing the effect of the control input on the state.
- $\boldsymbol{\omega}_t$: Gaussian process noise, with zero mean and covariance Q_t.

4.2.2 Observation Model

The observation model describes the relationship between the system state and the sensor measurements:

$$\mathbf{z}_t = C_t\mathbf{x}_t + \boldsymbol{v}_t$$

Where:

- $\mathbf{z}_t$: Sensor measurement.
- C_t: Matrix relating the state variables to the sensor measurements.
- $\mathbf{x}_t$: Current state vector.
- $\boldsymbol{v}_t$: Gaussian measurement noise, with zero mean and covariance R_t.

4.3 Variance and Covariance

4.3.1 Variance

Variance measures how much the values of a variable differ from their mean. For a set of N observations, the variance is:

$$\sigma_x^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2$$

Where $\bar{x}$ represents the mean of the observations.

4.3.2 Covariance

Covariance measures how two variables change together:

$$\sigma_{xy} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})$$

4.3.3 Covariance Matrix

The covariance matrix mathematically defines the uncertainty of the measurement and the covariance between two axes. For two variables, it is defined as:

$$R_t = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

The diagonal elements represent the variances, while the off-diagonal elements represent the covariance

4.4 Gaussian Probability Distributions

Assuming Gaussian noise, the probability distribution of the state transition model can be written as:

$$p(\mathbf{x}_t \mid \mathbf{u}_t, \mathbf{x}_{t-1}) = \det(2\pi Q_t)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}\mathbf{e}_t^T Q_t^{-1}\mathbf{e}_t\right)$$

Where the prediction error is:

$$\mathbf{e}_t = \mathbf{x}_t - A_t\mathbf{x}_{t-1} - B_t\mathbf{u}_t$$

Similarly, the measurement probability distribution is:

$$p(\mathbf{z}_t \mid \mathbf{x}_t) = \det(2\pi R_t)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{z}_t - C_t\mathbf{x}_t)^T R_t^{-1}(\mathbf{z}_t - C_t\mathbf{x}_t)\right)$$

The initial belief must be Gaussian so that the Gaussian assumption is maintained throughout the estimation process.

5 Derivation of the Kalman Filter by using the Recursive Least Squares Method

5.1 Introduction to the Least Squares Method

Consider a vehicle moving from an initial position y_0, with an initial velocity V_0 and an initial acceleration a. The objective is to estimate y_0, V_0 and a using only position measurements $y(t)$ obtained from a distance sensor. The displacement of an object is:

$$d = V_i t + \frac{1}{2}at^2$$

Therefore, the kinematic model describing the vehicle motion is:

$$\boxed{y(t) = y_0 + V_0 t + \frac{1}{2}at^2}$$

Since the model is continuous, we discretize it using a sampling interval Δt. Let:

$$t = k\Delta t$$

where k represents the measurement index. Substituting into the kinematic equation:

$$y_k = y_0 + V_0(k\Delta t) + \frac{1}{2}a(k\Delta t)^2$$

This expression can be written in matrix form:

$$y_k = \begin{bmatrix} 1 & k\Delta t & \frac{1}{2}(k\Delta t)^2 \end{bmatrix} \begin{bmatrix} y_0 \\ V_0 \\ a \end{bmatrix} + v_k$$

The measurement matrix is therefore:

$$C_k = \begin{bmatrix} 1 & k\Delta t & \frac{1}{2}(k\Delta t)^2 \end{bmatrix}$$

The state vector is:

$$\mathbf{x} = \begin{bmatrix} y_0 \\ V_0 \\ a \end{bmatrix}$$

The objective of the recursive least squares method as given in [4] is to estimate the state vector $\mathbf{x}$ using the measurements obtained over time:

$$y_0, y_1, y_2, \ldots$$

Whenever a new measurement arrives, the previous estimate is updated recursively.

5.2 Measurement Model

At instant k, the sensor measurement is:

$$\boxed{y_k = C_k\mathbf{x} + v_k}$$

We assume that the measurement noise v_k satisfies:

$$E[v_k] = 0$$

$$E[v_k v_k^T] = R_k$$

Where R_k is the measurement noise covariance matrix. The expected value represents the average of a random variable.

5.3 Recursive Least Squares Update

The recursive least squares method updates the previous state estimate using the new measurement:

$$\boxed{\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k-1} + K_k(y_k - C_k\hat{\mathbf{x}}_{k-1})}$$

Where:

- $\hat{\mathbf{x}}_k$: Estimated state vector at instant k.
- $\hat{\mathbf{x}}_{k-1}$: Previous state estimate.
- K_k: Gain matrix.
- y_k: Current measurement.
- C_k: Measurement matrix.

This equation updates the previous information using the measurement residual and the gain matrix K_k. The objective is to derive an expression for K_k by minimizing the estimation error.

5.4 Estimation Error

The estimation error vector is defined as:

$$\boldsymbol{\epsilon}_k = \mathbf{x} - \hat{\mathbf{x}}_k$$

For an n-dimensional state vector:

$$\boldsymbol{\epsilon}_k = \begin{bmatrix} \epsilon_{1,k} \\ \epsilon_{2,k} \\ \vdots \\ \epsilon_{n,k} \end{bmatrix} = \begin{bmatrix} x_1 - \hat{x}_{1,k} \\ x_2 - \hat{x}_{2,k} \\ \vdots \\ x_n - \hat{x}_{n,k} \end{bmatrix}$$

For the vehicle example:

$$\boldsymbol{\epsilon}_k = \begin{bmatrix} y_0 - \hat{y}_{0,k} \\ V_0 - \hat{V}_{0,k} \\ a - \hat{a}_k \end{bmatrix}$$

5.5 Cost Function

The gain matrix K_k is obtained by minimizing the sum of the expected squared estimation errors:

$$w_k = E[\epsilon_{1,k}^2] + E[\epsilon_{2,k}^2] + \cdots + E[\epsilon_{n,k}^2]$$

The estimation error covariance matrix is:

$$P_k = E[\boldsymbol{\epsilon}_k \boldsymbol{\epsilon}_k^T]$$

The cost function can therefore be expressed using the trace of the covariance matrix:

$$w_k = \mathrm{tr}(P_k)$$

The gain matrix is obtained by minimizing this cost function:

$$\frac{\partial w_k}{\partial K_k} = 0$$

6 Derivation of the Kalman Gain

The objective of the Kalman filter is to minimize the estimation error covariance matrix. We define the cost function as the trace of the covariance matrix:

$$W_k = \mathrm{tr}(P_k)$$

where the estimation error covariance matrix is:

$$P_k = E[\varepsilon_k \varepsilon_k^T]$$

6.1 Propagation of the Estimation Error

First, we define the estimation error:

$$\begin{aligned}\varepsilon_k &= x - \hat{x}_k \\ &= x - \hat{x}_{k-1} - K_k(y_k - C_k\hat{x}_{k-1}) \\ &= x - \hat{x}_{k-1} - K_k(C_k x + v_k - C_k\hat{x}_{k-1}) \\ &= x - \hat{x}_{k-1} - K_k C_k x - K_k v_k + K_k C_k \hat{x}_{k-1}\end{aligned}$$

Factoring the expression:

$$\varepsilon_k = (I - K_k C_k)(x - \hat{x}_{k-1}) - K_k v_k$$

Since the previous estimation error is:

$$\varepsilon_{k-1} = x - \hat{x}_{k-1}$$

we obtain:

$$\varepsilon_k = (I - K_k C_k)\varepsilon_{k-1} - K_k v_k$$

6.2 Propagation of the Estimation Error Outer Product

We calculate the outer product of the estimation error:

$$\varepsilon_k \varepsilon_k^T = \left[(I - K_k C_k)\varepsilon_{k-1} - K_k v_k\right]\left[(I - K_k C_k)\varepsilon_{k-1} - K_k v_k\right]^T$$

We apply the transpose property:

$$(A - B)^T = A^T - B^T$$

Therefore:

$$\varepsilon_k \varepsilon_k^T = \left[(I - K_k C_k)\varepsilon_{k-1} - K_k v_k\right]\left[\varepsilon_{k-1}^T (I - K_k C_k)^T - v_k^T K_k^T\right]$$

Applying the distributive property:

$$(A + B)(C + D) = AC + AD + BC + BD$$

we obtain:

$$\varepsilon_k \varepsilon_k^T = (I - K_k C_k)\varepsilon_{k-1}\varepsilon_{k-1}^T (I - K_k C_k)^T - (I - K_k C_k)\varepsilon_{k-1} v_k^T K_k^T$$
$$- K_k v_k \varepsilon_{k-1}^T (I - K_k C_k)^T + K_k v_k v_k^T K_k^T$$

6.3 Propagation of the Covariance Matrix

Recall that:

$$P_k = E[\varepsilon_k \varepsilon_k^T]$$

Since the measurement noise at time k, v_k, and the estimation error at time $k-1$, ε_{k-1}, are assumed to be independent, we have:

$$E[\varepsilon_{k-1} v_k^T] = 0$$

$$E[v_k \varepsilon_{k-1}^T] = 0$$

Additionally, we know that:

$$E[v_k v_k^T] = R_k$$

and

$$E[\varepsilon_{k-1} \varepsilon_{k-1}^T] = P_{k-1}$$

Therefore, the covariance matrix propagation equation becomes:

$$\boxed{P_k = (I - K_k C_k) P_{k-1} (I - K_k C_k)^T + K_k R_k K_k^T}$$

6.4 Expansion of the Covariance Matrix

To obtain the optimal Kalman Gain, we expand the covariance matrix expression:

$$\begin{aligned} P_k &= (I - K_k C_k) P_{k-1} (I - K_k C_k)^T + K_k R_k K_k^T \\ &= (I - K_k C_k) P_{k-1} (I - C_k^T K_k^T) + K_k R_k K_k^T \\ &= P_{k-1} - P_{k-1} C_k^T K_k^T - K_k C_k P_{k-1} + K_k C_k P_{k-1} C_k^T K_k^T + K_k R_k K_k^T \end{aligned}$$

6.5 Substitution into the Cost Function

Recall that our objective is to minimize:

$$W_k = \mathrm{tr}(P_k)$$

Substituting the expanded covariance matrix:

$$\begin{aligned} W_k = \ & \operatorname{tr}(P_{k-1}) - \operatorname{tr}(P_{k-1}C_k^T K_k^T) - \operatorname{tr}(K_k C_k P_{k-1}) \\ & + \operatorname{tr}(K_k C_k P_{k-1} C_k^T K_k^T) + \operatorname{tr}(K_k R_k K_k^T) \end{aligned}$$

We find the optimal matrix K_k by minimizing the cost function. Therefore, we calculate its partial derivative with respect to the Kalman Gain and set it equal to zero:

$$\frac{\partial W_k}{\partial K_k} = 0$$

6.6 Matrix Derivative Properties

Before differentiating the cost function, we recall the following matrix derivative properties:

$$1) \quad \frac{\partial \operatorname{tr}(AX^T B)}{\partial X} = BA$$

$$2) \quad \frac{\partial \operatorname{tr}(AXB)}{\partial X} = A^T B^T$$

$$3) \quad \frac{\partial \operatorname{tr}(XBX^T)}{\partial X} = XB^T + XB$$

$$4) \quad \frac{\partial \operatorname{tr}(P_{k-1})}{\partial K_k} = 0$$

The last property follows because P_{k-1} is constant with respect to K_k.

6.7 Applying the Derivative Properties

We differentiate each term of the cost function separately.

First term:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(P_{k-1}C_k^T K_k^T) = P_{k-1}C_k^T$$

Second term:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(K_k C_k P_{k-1}) = P_{k-1}C_k^T$$

Third term: Applying the third derivative property:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(K_k C_k P_{k-1} C_k^T K_k^T) = K_k C_k P_{k-1}^T C_k^T + K_k C_k P_{k-1} C_k^T$$

Since the covariance matrix is symmetric:

$$P_{k-1}^T = P_{k-1}$$

we obtain:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(K_k C_k P_{k-1} C_k^T K_k^T) = 2K_k C_k P_{k-1} C_k^T$$

Fourth term: Similarly:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(K_k R_k K_k^T) = K_k R_k^T + K_k R_k$$

Since the measurement noise covariance matrix is symmetric:

$$R_k^T = R_k$$

we obtain:

$$\frac{\partial}{\partial K_k} \operatorname{tr}(K_k R_k K_k^T) = 2K_k R_k$$

6.8 Setting the Derivative Equal to Zero

Combining all the derivatives:

$$0 = -P_{k-1} C_k^T - P_{k-1} C_k^T + 2K_k C_k P_{k-1} C_k^T + 2K_k R_k$$

Simplifying:

$$-2P_{k-1} C_k^T + 2K_k C_k P_{k-1} C_k^T + 2K_k R_k = 0$$

6.9 Factoring and Solving for the Kalman Gain

First, we move the negative term to the right-hand side:

$$2K_k C_k P_{k-1} C_k^T + 2K_k R_k = 2P_{k-1} C_k^T$$

Now, we factor out $2K_k$:

$$2K_k (C_k P_{k-1} C_k^T + R_k) = 2P_{k-1} C_k^T$$

Dividing both sides by 2:

$$K_k (C_k P_{k-1} C_k^T + R_k) = P_{k-1} C_k^T$$

Finally, multiplying both sides by the inverse of the matrix in parentheses:

$$\boxed{K_k = P_{k-1}C_k^T(C_kP_{k-1}C_k^T + R_k)^{-1}}$$

6.10 Final Kalman Filter Equations

In summary, we obtain the three equations that define the measurement update of the Kalman Filter:

1. Kalman Gain Update:

$$\boxed{K_k = P_{k-1}C_k^T(C_kP_{k-1}C_k^T + R_k)^{-1}}$$

2. State Estimate Update:

$$\boxed{\hat{x}_k = \hat{x}_{k-1} + K_k(y_k - C_k\hat{x}_{k-1})}$$

3. Propagation of the Estimation Error Covariance Matrix:

$$\boxed{P_k = (I - K_kC_k)P_{k-1}(I - K_kC_k)^T + K_kR_kK_k^T}$$

Alternatively, using the optimal Kalman Gain, the covariance matrix can also be expressed as:

$$\boxed{P_k = (I - K_kC_k)P_{k-1}}$$

6.11 Prediction and Correction Steps of the Kalman Filter

The Kalman Filter operates in two main steps:

1. **Prediction (A Priori):** Before receiving the measurement, the state estimate is predicted using the system's dynamic model. This provides the a priori estimate $\hat{x}_k^-$.
2. **Correction (A Posteriori):** After receiving the measurement y_k, the prediction is corrected using the measurement information. This provides the a posteriori estimate $\hat{x}_k^+$.

Since the covariance matrix P_k represents the uncertainty of the estimation, we distinguish between the a priori and a posteriori covariance matrices:

$$P_k^- = E\left[(x_k - \hat{x}_k^-)(x_k - \hat{x}_k^-)^T\right]$$

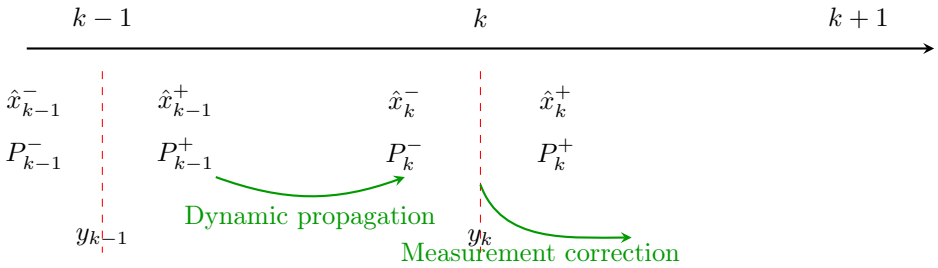
$$P_k^+ = E\left[(x_k - \hat{x}_k^+)(x_k - \hat{x}_k^+)^T\right]$$

6.11.1 Kalman Filter Flow Diagram

Regarding the Kalman Filter, we have seen that it operates in two main steps:

- **Prediction:** A Priori, before receiving the measurement, we calculate the predicted state estimate and covariance matrix.
- **Correction:** A Posteriori, after receiving the measurement, we calculate the corrected state estimate and covariance matrix.

The sequence of operations can be represented as follows:



Before each red line, at instant $k-1$, the estimate is the result of the previous measurement correction. At the a priori stage, the dynamic model propagates the previous state estimate $\hat{x}_{k-1}^+$ and covariance matrix P_{k-1}^+.

At the a posteriori stage, once the measurement y_k arrives, the Kalman Filter applies the measurement correction to calculate the Kalman Gain K_k and update both the state estimate and covariance matrix.

6.11.2 Initialization of the Prediction Step

To initialize the Kalman Filter, we begin at instant $k = 0$. We assume that the initial state is known, meaning that:

$$\hat{x}_0^+ = x_0$$

Therefore, the a priori estimate at instant $k = 1$ is obtained by propagating the initial state through the system's dynamic model:

$$\hat{x}_1^- = A_0\hat{x}_0^+ + B_0u_0$$

6.11.3 Prediction Step (A Priori)

Before receiving the measurement y_k, we propagate the covariance matrix using the system dynamics.

Recall the linear dynamic model:

$$\mathbf{x}_k = A_{k-1}\mathbf{x}_{k-1} + B_{k-1}\mathbf{u}_{k-1} + \mathbf{w}_{k-1}$$

where $\mathbf{w}_{k-1}$ is the process noise, assumed zero-mean and independent of the previous estimation error, with covariance

$$E[\mathbf{w}_{k-1}\mathbf{w}_{k-1}^T] = Q_{k-1}.$$

The a priori prediction of the mean is obtained by propagating the previous estimate through the dynamic model, without the process noise:

$$\hat{\mathbf{x}}_k^- = A_{k-1}\hat{\mathbf{x}}_{k-1}^+ + B_{k-1}\mathbf{u}_{k-1}$$

The prediction error is defined as:

$$\mathbf{e}_k^- = \mathbf{x}_k - \hat{\mathbf{x}}_k^-.$$

Substituting the dynamic model and the predicted mean:

$$\begin{aligned}\mathbf{e}_k^- &= (A_{k-1}\mathbf{x}_{k-1} + B_{k-1}\mathbf{u}_{k-1} + \mathbf{w}_{k-1}) - \left(A_{k-1}\hat{\mathbf{x}}_{k-1}^+ + B_{k-1}\mathbf{u}_{k-1}\right) \\ &= A_{k-1}\mathbf{x}_{k-1} - A_{k-1}\hat{\mathbf{x}}_{k-1}^+ + \mathbf{w}_{k-1} \\ &= A_{k-1}\left(\mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1}^+\right) + \mathbf{w}_{k-1}.\end{aligned}$$

Since the previous estimation error is

$$\mathbf{e}_{k-1}^+ = \mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1}^+,$$

we obtain:

$$\mathbf{e}_k^- = A_{k-1}\mathbf{e}_{k-1}^+ + \mathbf{w}_{k-1}$$

The a priori covariance matrix is defined as:

$$P_k^- = E\left[\mathbf{e}_k^-\left(\mathbf{e}_k^-\right)^T\right].$$

Substituting the expression for $\mathbf{e}_k^-$:

$$\begin{aligned}P_k^- &= E\left[\left(A_{k-1}\mathbf{e}_{k-1}^+ + \mathbf{w}_{k-1}\right)\left(A_{k-1}\mathbf{e}_{k-1}^+ + \mathbf{w}_{k-1}\right)^T\right] \\ &= E\Big[A_{k-1}\mathbf{e}_{k-1}^+\left(\mathbf{e}_{k-1}^+\right)^T A_{k-1}^T + A_{k-1}\mathbf{e}_{k-1}^+\mathbf{w}_{k-1}^T \\ &\qquad + \mathbf{w}_{k-1}\left(\mathbf{e}_{k-1}^+\right)^T A_{k-1}^T + \mathbf{w}_{k-1}\mathbf{w}_{k-1}^T\Big].\end{aligned}$$

Using the linearity of the expectation and taking the deterministic matrices A_{k-1} and A_{k-1}^T outside:

$$\begin{aligned}P_k^- = {} & A_{k-1}E\left[\mathbf{e}_{k-1}^+\left(\mathbf{e}_{k-1}^+\right)^T\right]A_{k-1}^T \\ & + A_{k-1}E\left[\mathbf{e}_{k-1}^+\mathbf{w}_{k-1}^T\right] + E\left[\mathbf{w}_{k-1}\left(\mathbf{e}_{k-1}^+\right)^T\right]A_{k-1}^T \\ & + E\left[\mathbf{w}_{k-1}\mathbf{w}_{k-1}^T\right].\end{aligned}$$

By definition,

$$E\left[\mathbf{e}_{k-1}^+\left(\mathbf{e}_{k-1}^+\right)^T\right] = P_{k-1}^+,$$

and

$$E\left[\mathbf{w}_{k-1}\mathbf{w}_{k-1}^T\right] = Q_{k-1}.$$

Since the process noise $\mathbf{w}_{k-1}$ is independent of the previous estimation error $\mathbf{e}_{k-1}^+$, the cross terms vanish:

$$E\left[\mathbf{e}_{k-1}^+\mathbf{w}_{k-1}^T\right] = E\left[\mathbf{e}_{k-1}^+\right]E\left[\mathbf{w}_{k-1}^T\right] = 0,$$

$$E\left[\mathbf{w}_{k-1}\left(\mathbf{e}_{k-1}^+\right)^T\right] = E\left[\mathbf{w}_{k-1}\right]E\left[\left(\mathbf{e}_{k-1}^+\right)^T\right] = 0.$$

Therefore, the a priori covariance matrix is:

$$\boxed{P_k^- = A_{k-1}P_{k-1}^+A_{k-1}^T + Q_{k-1}}$$

The term $B_{k-1}\mathbf{u}_{k-1}$ does not appear in the covariance propagation because the control input is assumed to be known and deterministic. It shifts the mean of the prediction but does not add uncertainty. If the

control input were random, an additional term $B_{k-1}\Sigma_u B_{k-1}^T$ would appear.

For the initial prediction, we assume:

$$P_0^+ = 0 \cdot I$$

Therefore, the first a priori covariance matrix is calculated as:

$$\boxed{P_1^- = A_0 P_0^+ A_0^T + Q_0}$$

6.11.4 Correction Step (A Posteriori)

Once the measurement y_k arrives, we perform the correction using the three equations previously derived.

1. Kalman Gain Update:

$$\boxed{K_k = P_k^- C_k^T (R_k + C_k P_k^- C_k^T)^{-1}}$$

2. State Estimate Update:

$$\boxed{\hat{x}_k^+ = \hat{x}_k^- + K_k(y_k - C_k \hat{x}_k^-)}$$

3. Propagation of the Estimation Error Covariance Matrix:

$$P_k^+ = (I - K_k C_k) P_k^- (I - K_k C_k)^T + K_k R_k K_k^T$$

Alternatively, using the optimal Kalman Gain:

$$\boxed{P_k^+ = (I - K_k C_k) P_k^-}$$

6.11.5 Notation Summary

We use the following notation to distinguish between the values before and after receiving the measurement y_k:

$P_{k-1} \longrightarrow P_{k-1}^+$	Before and after the previous measurement
$P_k \longrightarrow P_k^+$	After receiving y_k
$\hat{x}_{k-1} \longrightarrow \hat{x}_{k-1}^+$	After the previous measurement
$\hat{x}_k \longrightarrow \hat{x}_k^+$	After receiving y_k

6.12 Final Kalman Filter equations

$$\boxed{\begin{aligned}
&(1) \quad K_i = P_i^- C_i^T \left(R_i + C_i P_i^- C_i^T\right)^{-1} \\
&(2) \quad \hat{x}_i^+ = \hat{x}_i^- + K_i(y_i - C_i \hat{x}_i^-) \\
&(3) \quad P_i^+ = (I - K_i C_i) P_i^- (I - K_i C_i)^T + K_i R_i K_i^T \\
&\qquad\quad = (I - K_i C_i) P_i^-
\end{aligned}} \quad \Big\} \textbf{ Correction}$$

$$\boxed{\begin{aligned}&(1)\quad \hat{x}_{i+1}^{-} = A_i \hat{x}_i^{+} + B_i u_i \\ &(2)\quad P_{i+1}^{-} = A_i P_i^{+} A_i^T + Q_i\end{aligned}} \Bigg\} \textbf{Prediction}$$

7 Extended Kalman Filter

7.1 Motivation

The standard Kalman Filter assumes that the system is linear and that the state uncertainty follows a Gaussian distribution. However, real systems are not always linear.

When applying a nonlinear transformation to a Gaussian distribution, the resulting distribution is not necessarily Gaussian. The transformation can deform the original probability distribution, making it asymmetric or changing its shape.

This is illustrated below.

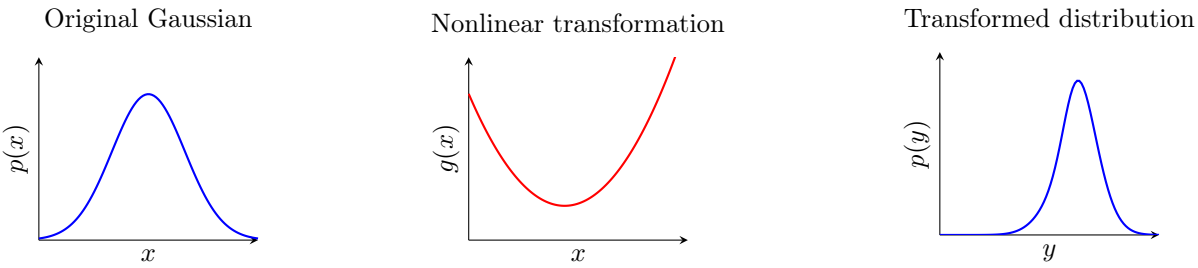


Figure 2: Illustration of how a nonlinear transformation can deform a Gaussian probability distribution.

The Extended Kalman Filter (EKF) addresses this problem by approximating nonlinear functions using a first-order Taylor expansion around the current state estimate.

This local linearization allows us to propagate the mean and covariance while maintaining an approximately Gaussian representation of the uncertainty.

7.2 Nonlinear System Model

The general nonlinear system is described by two equations:

$$\begin{aligned} x_t &= g(u_t, x_{t-1}) + \epsilon_t \\ z_t &= h(x_t) + \delta_t \end{aligned}$$

Where:

- x_t: system state.
- u_t: control input.
- z_t: measurement.
- $g(\cdot)$: nonlinear process model.
- $h(\cdot)$: nonlinear measurement model.
- ϵ_t: process noise.

- δ_t: measurement noise.

For comparison, the equivalent linear system would be:

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$
$$z_t = C_t x_t + \delta_t$$

Therefore, we need to approximate the nonlinear functions using first-order Taylor expansions.

7.3 First-Order Taylor Approximation

7.3.1 1. Prediction

We approximate the nonlinear process function around the previous posterior state estimate:

$$g(u_t, x_{t-1}) \approx g(u_t, \hat{x}_{t-1}^+) + \left.\frac{\partial g(u_t, x)}{\partial x}\right|_{\hat{x}_{t-1}^+} (x_{t-1} - \hat{x}_{t-1}^+)$$

Where:

- x_{t-1} represents the real state.
- $\hat{x}_{t-1}^+$ represents the previous posterior estimate.

We define the Jacobian of the process model as:

$$G_t = \left.\frac{\partial g(u_t, x)}{\partial x}\right|_{\hat{x}_{t-1}^+}$$

Thus, the linearized process model becomes:

$$x_t \approx g(u_t, \hat{x}_{t-1}^+) + G_t(x_{t-1} - \hat{x}_{t-1}^+) + \epsilon_t$$

7.3.2 2. Correction

Similarly, we approximate the nonlinear measurement function around the prior state estimate:

$$h(x_t) \approx h(\hat{x}_t^-) + \left.\frac{\partial h(x)}{\partial x}\right|_{\hat{x}_t^-} (x_t - \hat{x}_t^-)$$

Where:

- x_t represents the real state.
- $\hat{x}_t^-$ represents the prior state estimate.

We define the Jacobian of the measurement model as:

$$H_t = \left.\frac{\partial h(x)}{\partial x}\right|_{\hat{x}_t^-}$$

Therefore, the linearized measurement model becomes:

$$z_t \approx h(\hat{x}_t^-) + H_t(x_t - \hat{x}_t^-) + \delta_t$$

Both Jacobians, G_t and H_t, play the same role as the matrices A_t and C_t in the standard Kalman Filter.

7.3.3 Geometrical Interpretation

The EKF approximates a nonlinear function using its tangent line at the current estimate. This approximation is accurate locally, but it may become less accurate as the uncertainty increases [5].

The following diagrams illustrate the approximation of probability distributions.

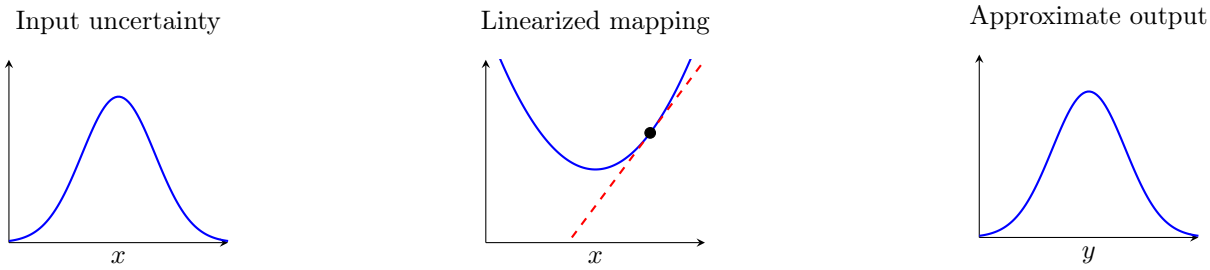


Figure 3: The EKF propagates uncertainty through a local linear approximation, obtaining an approximately Gaussian output distribution.

Although this approximation is not perfect, it provides a practical way to estimate nonlinear systems. The smaller the uncertainty, the more accurate the local approximation tends to be.

7.4 Extended Kalman Filter Algorithm

The EKF follows the same two main stages as the standard Kalman Filter:

1. Prediction: estimate the next state and its covariance before incorporating the new measurement.
2. Correction: incorporate the new measurement to update the state estimate and its covariance.

The algorithm receives the previous posterior estimate, its covariance, the control input, and the new measurement:

$$\text{EKF}\left(\hat{x}_{k-1}^+, P_{k-1}^+, u_k, y_k\right)$$

7.4.1 1. Prediction Step

First, predict the next state using the nonlinear process model:

$$\boxed{\hat{x}_k^- = g(u_k, \hat{x}_{k-1}^+)}$$

Then, propagate the covariance matrix using the process Jacobian:

$$\boxed{P_k^- = G_k P_{k-1}^+ G_k^T + Q_k}$$

Where G_k is evaluated at the previous posterior state estimate.

7.4.2 2. Correction Step

Once the new measurement y_k becomes available, calculate the Kalman Gain:

$$\boxed{K_k = P_k^- H_k^T \left(H_k P_k^- H_k^T + R_k\right)^{-1}}$$

Update the state estimate using the measurement residual:

$$\boxed{\hat{x}_k^+ = \hat{x}_k^- + K_k \left(y_k - h(\hat{x}_k^-)\right)}$$

Finally, update the estimation error covariance matrix:

$$\boxed{P_k^+ = (I - K_k H_k) P_k^-}$$

7.4.3 3. Return

The algorithm returns the posterior state estimate and its covariance:

$$\boxed{\text{return } \left(\hat{x}_k^+, P_k^+\right)}$$

7.5 Summary of the EKF Equations

The complete EKF algorithm can be summarized using the following five equations:

$$\begin{aligned}
\textbf{Prediction:} \quad & \hat{x}_k^- = g(u_k, \hat{x}_{k-1}^+) \\
& P_k^- = G_k P_{k-1}^+ G_k^T + Q_k \\
\textbf{Correction:} \quad & K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \\
& \hat{x}_k^+ = \hat{x}_k^- + K_k (y_k - h(\hat{x}_k^-)) \\
& P_k^+ = (I - K_k H_k) P_k^-
\end{aligned}$$

Notation:

- G_k: Jacobian of the nonlinear process model.
- H_k: Jacobian of the nonlinear measurement model.
- Q_k: process noise covariance matrix.
- R_k: measurement noise covariance matrix.
- $(-)$: prior estimate, before incorporating the measurement.
- $(+)$: posterior estimate, after incorporating the measurement.

8 Experimentation

8.1 Experimental Objective

The objective of this experiment is to implement an Extended Kalman Filter (EKF) to fuse information from two sensors in a mobile differential robot [6]:

- Wheel encoders: provide linear and angular velocities for state prediction.
- IMU: provides angular velocity around the Z-axis, which is integrated to obtain an orientation measurement for the correction step.

The state vector is defined as:

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

Where x_t and y_t represent the robot's position, and θ_t represents its orientation.

8.2 Motion Model: Wheel Encoders

The motion model describes the robot's displacement using the linear velocity v and angular velocity ω, obtained from the wheel encoders [7].

The state transition equation is:

$$\mathbf{x}_t = A\mathbf{x}_{t-1} + B\mathbf{u}_{t-1}$$

Where:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} \cos(\theta_{t-1})\Delta t & 0 \\ \sin(\theta_{t-1})\Delta t & 0 \\ 0 & \Delta t \end{bmatrix}$$

$$\mathbf{u}_{t-1} = \begin{bmatrix} v_{t-1} \\ \omega_{t-1} \end{bmatrix}$$

Expanding the matrix multiplication gives the nonlinear motion equations:

$$\boxed{\begin{aligned} x_t &= x_{t-1} + v_{t-1}\cos(\theta_{t-1})\Delta t \\ y_t &= y_{t-1} + v_{t-1}\sin(\theta_{t-1})\Delta t \\ \theta_t &= \theta_{t-1} + \omega_{t-1}\Delta t \end{aligned}}$$

8.2.1 State Transition Jacobian

To linearize the nonlinear motion model, we calculate the Jacobian matrix G.

Define the transition functions:

$$
\begin{aligned}
g_1 &= x_{t-1} + v\cos(\theta_{t-1})\Delta t \\
g_2 &= y_{t-1} + v\sin(\theta_{t-1})\Delta t \\
g_3 &= \theta_{t-1} + \omega\Delta t
\end{aligned}
$$

The Jacobian is:

$$
G = \begin{bmatrix}
\frac{\partial g_1}{\partial x} & \frac{\partial g_1}{\partial y} & \frac{\partial g_1}{\partial \theta} \\
\frac{\partial g_2}{\partial x} & \frac{\partial g_2}{\partial y} & \frac{\partial g_2}{\partial \theta} \\
\frac{\partial g_3}{\partial x} & \frac{\partial g_3}{\partial y} & \frac{\partial g_3}{\partial \theta}
\end{bmatrix}
$$

Therefore:

$$
G = \begin{bmatrix}
1 & 0 & -v\sin(\theta_{t-1})\Delta t \\
0 & 1 & v\cos(\theta_{t-1})\Delta t \\
0 & 0 & 1
\end{bmatrix}
$$

8.3 Measurement Model: IMU

The IMU provides angular velocity around the Z-axis through its gyroscope. Since the gyroscope does not directly measure orientation, the angular velocity measurements are integrated over time to obtain an orientation estimate.

The angular velocity measurement is corrected using the gyroscope bias:

$$
\omega_{z,t}^{\text{IMU}} = \omega_{z,t}^{\text{measured}} - b
$$

Where b represents the gyroscope bias.

The orientation estimate obtained from the IMU is calculated as:

$$
\theta_{\text{IMU},t} = \theta_{\text{IMU},t-1} + \omega_{z,t}^{\text{IMU}}\Delta t
$$

This integrated orientation is used as the measurement input during the EKF correction stage:

$$
z_t = \begin{bmatrix} \theta_{\text{IMU},t} \end{bmatrix}
$$

The measurement function relates the robot's state vector to the measured orientation:

$$
h(\mathbf{x}_t) = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}
$$

Therefore, the measurement function simplifies to:

$$
h(\mathbf{x}_t) = \theta_t
$$

The measurement Jacobian is obtained by differentiating the measurement function with respect to the state vector:

$$H_t = \frac{\partial h(\mathbf{x}_t)}{\partial \mathbf{x}_t}$$

Thus:

$$H_t = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

This measurement model allows the EKF to correct the predicted orientation using the orientation estimate obtained from the integrated gyroscope measurements, while the robot's position continues to be estimated through the encoder-based motion model.

8.4 Experimental Estimation of Covariance Matrices

The EKF requires two covariance matrices:

- Q: Process noise covariance.
- R: Measurement noise covariance.

8.4.1 Process Noise Covariance (Q)

The matrix Q represents uncertainty in the robot's motion model, particularly errors associated with the wheel encoders where σ_x^2 and σ_y^2 are measured in m^2 and σ_θ^2 in rad^2.

Assuming independent errors, the covariance matrix is:

$$Q = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix}$$

Experimental procedure To estimate the process noise, the Puzzlebot was commanded to perform controlled movements.

For position uncertainty:

1. Command the robot to move 1 meter according to the wheel encoders.
2. Measure the actual distance traveled.
3. Calculate the difference between the expected and actual distances.
4. Repeat the experiment to obtain representative error statistics.

For orientation uncertainty:

1. Command the robot to perform a 360-degree rotation.
2. Measure the actual angular displacement.
3. Compare the encoder-based angle with the actual angle.
4. Repeat the experiment to estimate the orientation uncertainty.

For example, if the encoders indicate a displacement of 1.00 m while the actual displacement is 0.98 m:

$$e_x = 1.00 - 0.98 = 0.02\,m$$

An initial variance approximation is:

$$\sigma_x^2 \approx (0.02)^2 = 0.0004\,m^2$$

Experimental results The experimental evaluation of the Puzzlebot resulted in the following process noise covariance matrix:

$$Q = \begin{bmatrix} 0.0004 & 0 & 0 \\ 0 & 0.00002 & 0 \\ 0 & 0 & 0.0157 \end{bmatrix}$$

The diagonal elements represent the estimated variances associated with the robot's position in the X and Y axes and its orientation, respectively.

8.4.2 Measurement Noise Covariance (R)

The matrix R represents the uncertainty associated with the IMU measurements.

Since only angular velocity around the Z-axis is used, the measurement noise covariance is:

$$R = \begin{bmatrix} \sigma_{\omega_z}^2 \end{bmatrix}$$

Experimental procedure To estimate the IMU measurement noise:

1. Keep the Puzzlebot completely stationary.
2. Collect 1000 angular velocity measurements from the IMU around the Z-axis.
3. Calculate the mean of the measurements.
4. Calculate their sample variance.

The sample variance is:

$$\sigma_{\omega_z}^2 = \frac{1}{N-1}\sum_{i=1}^{N}(\omega_{z,i} - \bar{\omega}_z)^2$$

Where $N = 1000$.

Experimental results After collecting and analyzing the IMU measurements, the following measurement noise covariance was obtained:

$$R = \begin{bmatrix} 0.000081 \end{bmatrix}$$

This value represents the experimentally estimated variance of the gyroscope's angular velocity measurements around the Z-axis.

9 Results

The EKF implementation was evaluated using a real differential-drive robot following two reference trajectories: a rectangular path and a zigzag path. The experimental results were compared against the odometry obtained exclusively from the wheel encoders. Three experimental runs were performed for each estimation method and trajectory. The robot's positions at the different waypoints were measured manually.

Figure 4 presents the individual trajectories obtained during the experiments. The first two rows correspond to the rectangular trajectory, while the last two rows show the zigzag trajectory. The red plots represent the encoder-only odometry results, whereas the blue plots represent the EKF estimates. These results illustrate the variations observed between experimental runs and provide a visual comparison of both estimation methods.

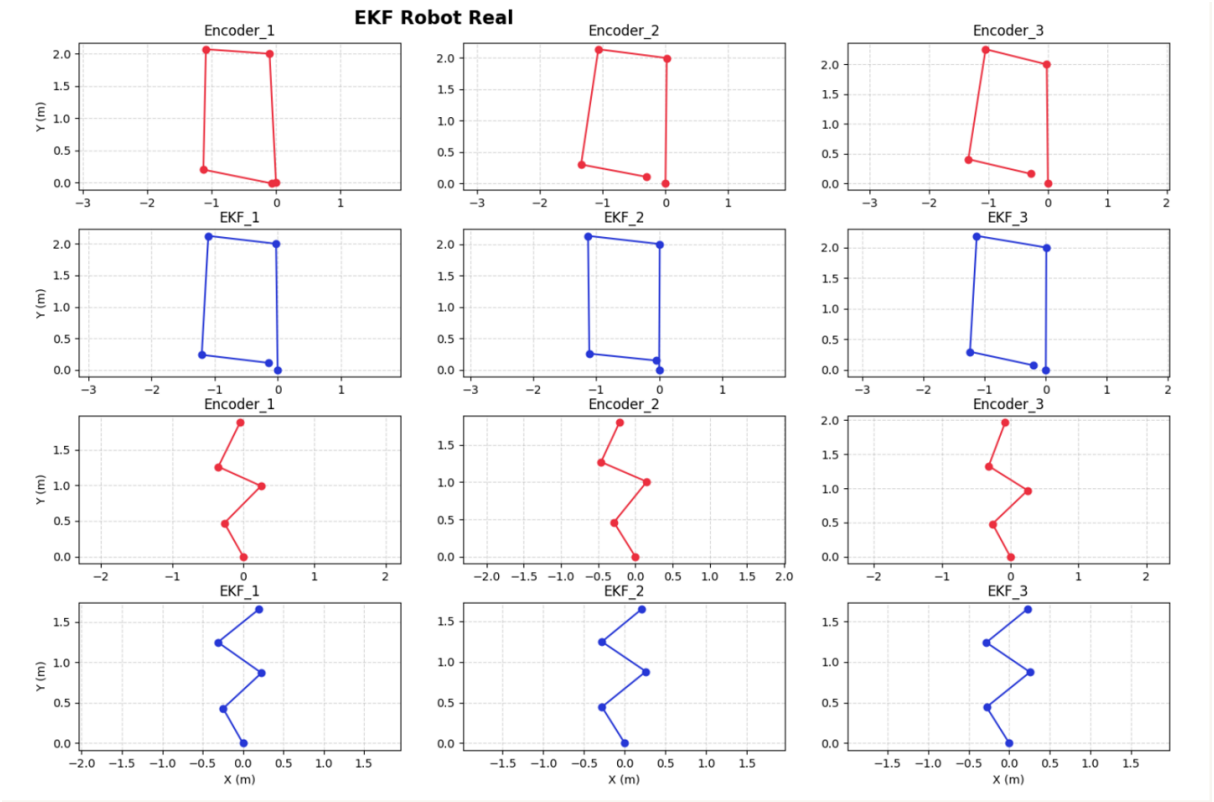


Figure 4: Individual rectangular and zigzag trajectories obtained using encoder-only odometry and the EKF on the real robot.

To facilitate the comparison, the mean trajectories obtained from the experimental runs were calculated and compared with the desired reference paths. As shown in Figure 5, each subplot presents three trajectories: the desired path, the encoder-only odometry, and the EKF estimation. This comparison provides a clearer representation of the overall deviation from the reference trajectory for both estimation methods.

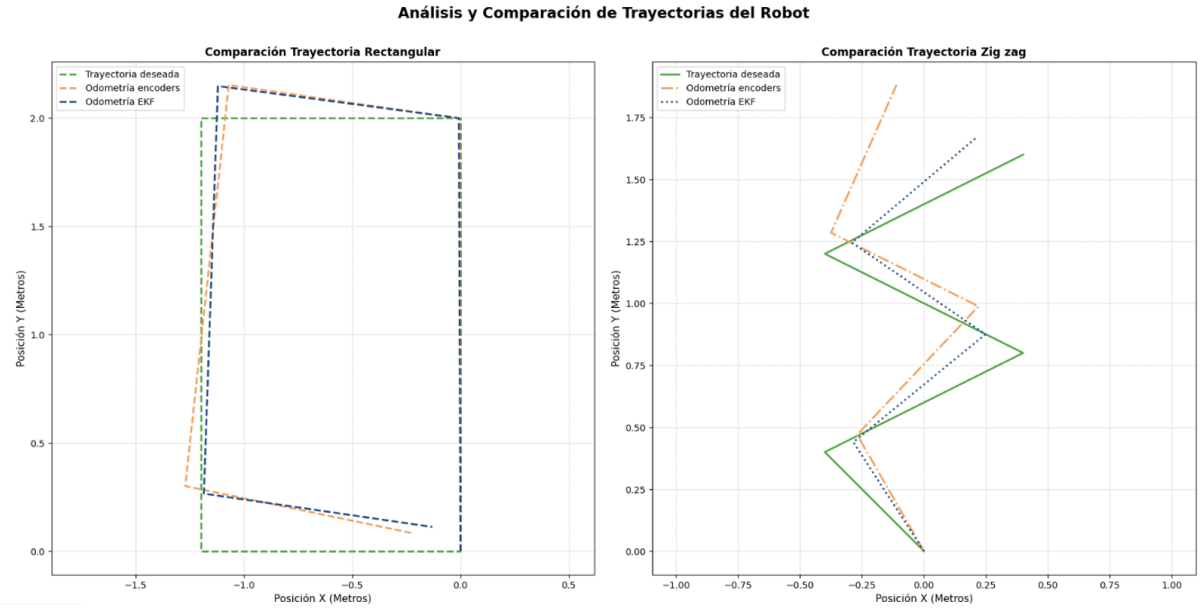


Figure 5: Comparison of the desired, encoder-only, and EKF mean trajectories for the rectangular and zigzag paths.

Finally, the positional error was evaluated at five control points along each trajectory, using the Euclidean distance between the measured robot position and the corresponding reference position. Figure 6 presents the comparison between encoder-only odometry and EKF estimation at each control point.

For the rectangular trajectory, the EKF reduced the final positional error by 27.01% compared with encoder-only odometry. The error decreased from approximately 0.235 m to 0.172 m at the final control point (P5). Although a small increase in error was observed at P2, the EKF achieved lower positional errors at the subsequent control points.

For the zigzag trajectory, the improvement was more significant. The final positional error decreased from approximately 0.582 m using encoder-only odometry to 0.201 m using the EKF, corresponding to a reported reduction of 65.41%. This result indicates that the EKF provided a greater improvement during the zigzag experiment, where changes in direction produced larger deviations in the encoder-only estimation.

Overall, the experimental results show that the EKF reduced the final positional error in both trajectories. The improvement was particularly noticeable in the zigzag experiment, demonstrating the potential benefit of incorporating IMU measurements into the robot's state estimation process.

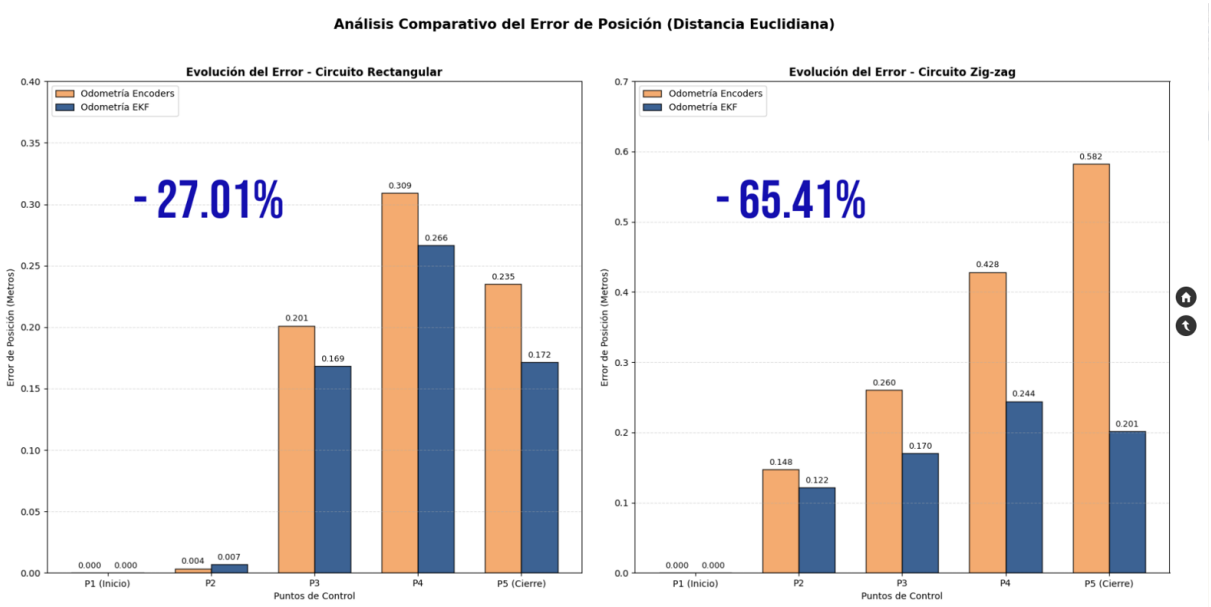


Figure 6: Comparison of positional errors at each control point for encoder-only odometry and EKF estimation.

10 Analysis

The experimental results indicate that the implementation of the Extended Kalman Filter improved the robot's localization performance compared with encoder-only odometry in both evaluated trajectories. However, the magnitude of this improvement varied depending on the path followed by the robot.

For the rectangular trajectory, the EKF reduced the final positional error from approximately 0.235 m to 0.172 m, corresponding to a reported reduction of 27.01%. Although the EKF produced a slightly higher error at the second control point (P2), it achieved lower errors at the subsequent points. This behavior suggests that the filter did not provide an improvement at every individual waypoint, but it reduced the accumulated positional deviation by the end of the trajectory.

A more significant improvement was observed during the zigzag experiment. The final positional error decreased from approximately 0.582 m with encoder-only odometry to 0.201 m with the EKF, representing a reported reduction of 65.41%. The larger improvement in this experiment may be related to the frequent changes in direction along the zigzag path. During these movements, small errors in the estimated angular displacement can produce considerable deviations in the robot's subsequent position. By incorporating the orientation estimate obtained from the integrated gyroscope measurements, the EKF was able to correct the predicted heading and consequently improve the propagation of the robot's planar position.

The comparison between the mean trajectories also shows that the EKF estimates followed the desired reference paths more closely than encoder-only odometry in the evaluated experiments. Nevertheless, some deviations remained, particularly toward the final control points. These errors may be associated with wheel slippage, encoder inaccuracies, gyroscope bias, and the accumulation of measurement noise over time.

It is also important to consider the limitations of the implemented estimation architecture. The gyroscope does not provide an absolute orientation measurement; instead, its angular velocity is integrated over time to obtain an orientation estimate. Consequently, errors and bias in the gyroscope measurements can accumulate and affect the correction stage. Furthermore, the experimental reference positions were measured manually, which introduces additional uncertainty into the positional error calculations.

Therefore, the reported improvements should be interpreted within the limitations of the experimental measurement procedure.

Overall, the results demonstrate that the EKF provided lower final positional errors than encoder-only odometry in both experiments, with a particularly noticeable improvement in the zigzag trajectory. This supports the use of sensor fusion to improve differential-drive robot localization, while also highlighting the importance of appropriate sensor calibration, noise characterization, and more precise ground-truth measurements in future experiments.

11 Conclusion

This work presented the mathematical derivation and experimental implementation of an Extended Kalman Filter (EKF) for planar pose estimation on a differential-drive robot. Starting from the Bayes filter and the Gaussian assumptions, the Kalman gain was derived by minimizing the trace of the estimation error covariance. The result was then extended to nonlinear systems through first-order linearization, which led to the Jacobians G_t and H_t used in the implementation.

The filter fused wheel-encoder odometry in the prediction stage with the orientation obtained by integrating the gyroscope's angular velocity in the correction stage. On the real robot, the EKF reduced the final positional error by 27.01% on the rectangular trajectory (0.235 m to 0.172 m) and by 65.41% on the zigzag trajectory (0.582 m to 0.201 m) compared with encoder-only odometry. The larger improvement on the zigzag path is consistent with heading errors having a greater impact when changes in direction are frequent.

These results should be interpreted with caution. Only three runs were performed per method, positions were measured manually, and the integrated gyroscope signal drifts over time because its bias is not part of the state. In addition, the noise covariances Q and R were obtained from simple calibration experiments rather than tuned systematically.

Future work includes augmenting the state vector with the gyroscope bias, incorporating an absolute heading or position source (e.g., a magnetometer, LiDAR, or camera), using an automated ground-truth system such as an overhead camera or motion capture, and increasing the number of runs to report means and standard deviations.

References

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[3] G. Welch and G. Bishop, "An introduction to the Kalman filter," Tech. Rep. TR 95-041, University of North Carolina at Chapel Hill, 1995.

[4] D. Simon, *Optimal State Estimation: Kalman, H_∞, and Nonlinear Approaches*. Hoboken, NJ: Wiley, 2006.

[5] CyrillStachniss, "Fltro de kalman y ekf." YouTube, `https://youtu.be/E-6paM_Iwfc?si=jeImnPOEa2vcC43n`, 2020. Accessed: 2026/05/10.

[6] Manchester Robotics, "Puzzlebot documentation." `https://github.com/ManchesterRoboticsLtd/MCR2`. Accessed: 2026/03/01.

[7] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. Cambridge, MA: MIT Press, 2 ed., 2011.